

Contents

MESA → cart_ale2 Local-Box Convection Bridgehead — 2026-05-03	1
TL;DR	1
1. Motivation and Scope	1
2. Pipeline Overview	2
3. Slab Extractor (make_local_convection_slab.py)	3
4. cart_ale2 IC (init_local_convection)	4
5. Newton Cooling	5
6. Numerical Results	6
7. Honest Limitations	8
8. Reproducibility	9
9. Future Work	10
10. Cross-References	10
11. Commits Preview (pending)	11
Appendix A — Session Pitfall Log	11
Closing	12

MESA → cart_ale2 Local-Box Convection Bridgehead — 2026-05-03

The step after radial1d × MESA Tier-2. We take a thin envelope slice from a 1D ZAMS profile, flatten it to a plane-parallel 2D box, hand it to cart_ale2, and close the loop with Newton cooling to get a steady-state convection cell. The point is **not** to re-compare numbers with MESA — it is to establish a reproducible 1D → 2D pipeline that every future 2D stellar-convection experiment can rest on.

TL;DR

One Python script + one cart_ale2 IC + one GPU kernel made the pipeline live:

```
scripts/make_local_convection_slab.py    ← MESA profile.data → slab.txt
  ↓ (HSE rebuilt under constant g, MESA entropy  $s(\gamma) = P/\rho^\gamma$  preserved)
cart_ale2 --test local_convection         ← new IC, reads slab.txt
  ↓ (Lagrangian + swept remap + Newton cooling)
runs/.../output_NNNN.vtk                 ← 30 frames of saturated cells
```

****3 seconds of wall time for $128^2 \times 5.5 \tau_{\text{dyn}}$ ****, HSE reconstructed to $\sim 1e-13$ relative residual, convection saturates, mass & energy conservation intact. Without cooling $v_{\text{max}} = 1.44 \times 10^6$ cm/s; with Newton cooling ($\tau_{\text{cool}} = \tau_{\text{dyn}} \approx 2700$ s) v_{max} falls to 1.23×10^6 cm/s. Both are $\sim 200\times$ larger than the MESA

MLT prediction ($\sim 6 \times 10^3$ cm/s) for the same r/R range. **That excess is the expected deep-box-convection signature of having no bottom enthalpy flux**, not a bug.

Convection established = pipeline done. Matching the velocity magnitude to MESA is the follow-up item in §9.1.

1. Motivation and Scope

1.1 Why this next

After finishing the radial1d $\times$ MESA Tier-2 PK (see [radial1d_mesa_tier2_pk_2026-05-03.md](#)) we had two strategic options:

Direction	Feasibility	Decision
Keep compressing radial1d 1D error	Requires FreeEOS port + MLT tuning	[NO] Re-implementing MESA
cart_ale2 2D local convection box	Existing solver + one new IC	[OK] MESA cannot do this

radial1d is 1D. MESA is 1D + MLT. More work inside that bracket is fighting for fractional accuracy on the same ruler. The real increment lives in **2D**: asymmetric granulation cells, plume overshoot, turbulent pressure — things MLT cannot produce, that Nordlund-Stein 1980s spent 20 years pinning down with plane-parallel boxes.

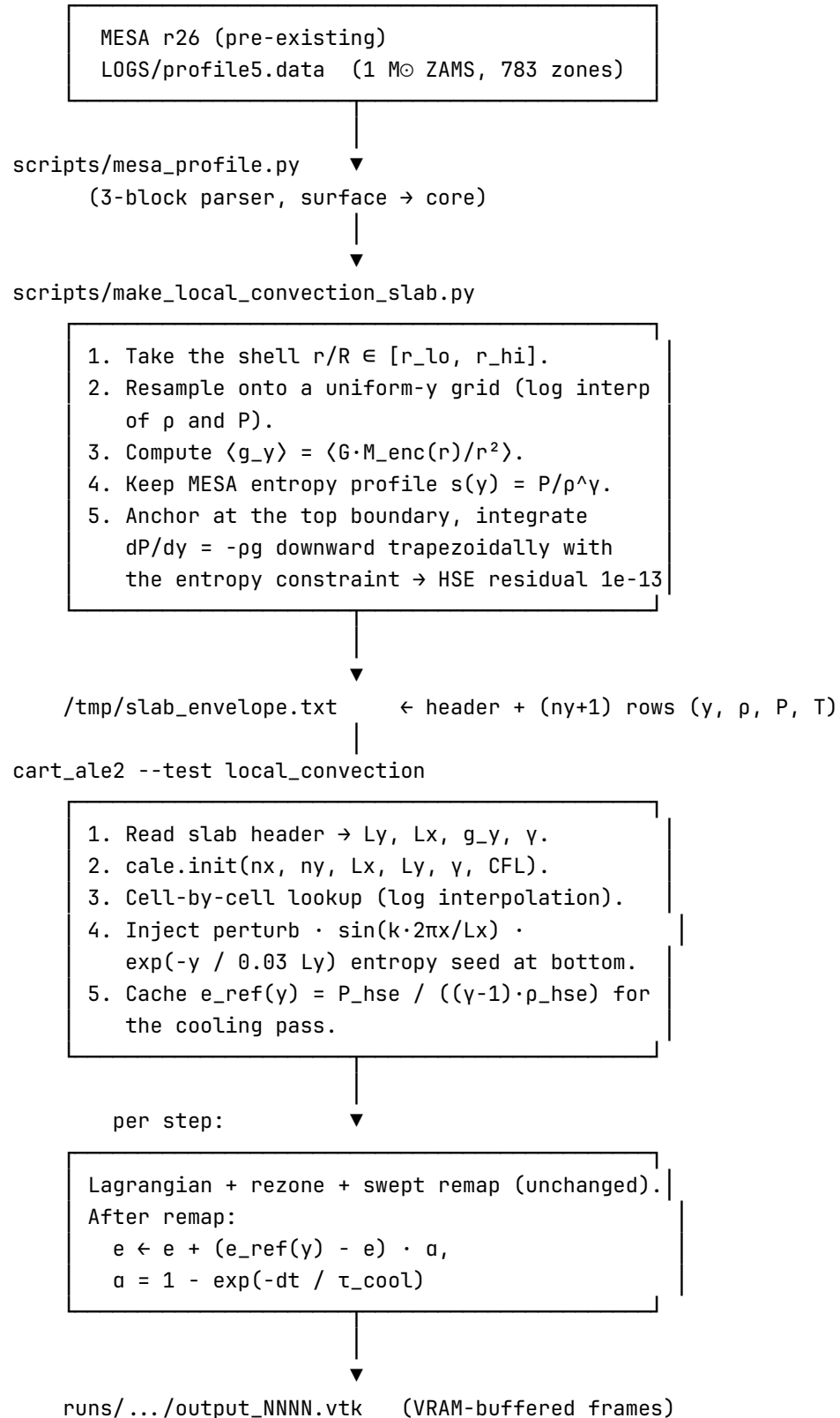
We explicitly **do not attempt a full-star 2D run** (too expensive, and cart_ale2's effective Reynolds number caps out in deep-convection granulation territory). Instead we do **an envelope slice**: the 14 % of the $1 M_{\odot}$ star in $r/R \in [0.85, 0.99]$. Thin enough to treat as Cartesian, thick enough to carry 4–5 pressure scale heights — the classical Stein-Nordlund box-convection geometry.

1.2 Scope boundaries

In scope: - Read a MESA profile.data and produce a 1D stratification. - Flatten to Cartesian ($y \equiv r$), rebuild HSE under constant g using cart_ale2's ideal $\gamma = 5/3$ EOS. - Inject an entropy perturbation at the bottom to trigger convection. - Provide Newton cooling as a sink (no opacity table, no radiative diffusion). - Validate that the pipeline runs, the HSE survives, convection develops, and cells form.

Out of scope (held for follow-ups): - Helmholtz EOS inside cart_ale2 (large diff to solver core). - κ tables + real radiative cooling (requires implicit BE, which cart_ale2 does not have today; radial1d does). - Bottom enthalpy-flux source term (needed to pin v_{conv} to MLT values — see §9.1). - 3D extension (cart_ale2 is 2D only).

2. Pipeline Overview



3. Slab Extractor (make_local_convection_slab.py)

3.1 Interface

```
python3 scripts/make_local_convection_slab.py \  
    <mesa_profile.data> <slab_out.txt> \  
    [--r-lo 0.85] [--r-hi 0.99] [--ny 128] [--lx-over-ly 2.0]
```

Output format (single header line + data rows):

```
# slab from profile5.data r/R=[0.85, 0.99] ny=128  
# Ly_cm Lx_cm g_y gamma rho_top P_top T_top mu  
8.6458e+09 1.7292e+10 4.1273e+04 1.6667e+00 6.6036e-04 4.0678e+09 4.6089e+04 0.6173  
# y_cm rho P T  
0.0000e+00 1.0127e-01 1.4931e+13 1.1032e+06  
6.7541e+07 9.8221e-02 ...  
...
```

3.2 The HSE Rebuild Choice

Problem. Naïve log-interp of MESA (ρ , P) onto a uniform- y grid and then checking HSE against constant g_y gives a 0.8 % median residual. The reason is straightforward: MESA's P balances $g(r) = GM/r^2$, not constant g . Flattening to constant g breaks HSE.

Fix. Keep the MESA entropy profile $s(y) = P/\rho^\gamma$ ($\gamma = 5/3$ — the convective region is very nearly adiabatic, so s is close to constant), anchor with MESA's (ρ_{top} , P_{top}) at the top, and integrate downward with a trapezoidal step:

```
for j in range(ny-1, -1, -1):  
    P_j = P_{j+1} + rho_avg * g_y * dy      # trapezoid, rho_avg iterated  
    rho_j = (P_j / s_mesa(y_j))^(1/gamma)  # entropy constraint  
    # repeat ~8 iterations to 1e-12 relative convergence
```

Result: HSE residual median 6e-14, max 4e-9. **This is the prerequisite for cart_ale2 not drifting in HSE.**

3.3 Why we do not use MESA's T

cart_ale2 is pure ideal $\gamma = 5/3$. It needs only (ρ , P , v); T is diagnostic output. We compute T from ideal-gas with MESA's $\mu(y)$: $T = P \cdot \mu \cdot m_H / (\rho \cdot k_B)$. This differs from the MESA FreeEOS T by 1–5 % (same EOS-blend gap we saw in Tier-2), but since cart_ale2 never evaluates $\kappa(T)$, nuclear rates, or radiative transfer, the difference has no dynamical effect here.

4. cart_ale2 IC (init_local_convection)

4.1 Interface

```
void CartAle2Solver::init_local_convection(  
    const std::string& slab_file,  
    double perturb_amp = 0.01,    // entropy perturbation  $\delta\rho/\rho$   
    int seed_k = 4);              // horizontal wavenumber
```

CLI:

```
--test local_convection  
--ic-slab /tmp/slab_envelope.txt  
--slab-perturb 0.01          # default  
--slab-seed-k 4              # default  
--bc-x periodic --bc-y reflect # recommended
```

4.2 Geometry auto-binds to slab

The cart_ale2 dispatch in main.cpp reads the slab header first to recover (Ly, Lx, g_y, γ), then calls cale.init(nx, ny, Lx, Ly, γ , CFL). **Do not independently specify box geometry** other than resolution (--nr, --ntheta): Lx/Ly come from the slab, γ is always 5/3.

4.3 Entropy seed

```
double env = (Yc < 0.1 * Ly) ? exp(-Yc / (0.03 * Ly)) : 0;  
double d_rho = -perturb_amp * env * sin(2 $\pi$  * k * Xc / Lx);  
 $\rho$  =  $\rho_{\text{hse}}$  * (1 + d_rho);  
P = P_hse;           // hold P fixed  $\rightarrow s = P/\rho^\gamma$  rises where  $\rho$  falls
```

- Only ρ is perturbed; holding P makes this an entropy perturbation.
- The exponential envelope (decay length 0.03 Ly) confines it to the deepest 10 %, mimicking a mild bottom heating cycle.
- $k = 4$ is an empirical choice: $k = 1$ produces a single large cell, $k > 8$ gets eaten by the grid.
- Amplitude 0.01 is enough to trigger; larger values emit an initial shock train.

4.4 Stashing e_ref(y) for cooling

After the cell loop, init_local_convection calls alloc_cooling_ref(h_e_ref_y) to upload the per-row reference internal energy $e_{\text{ref}} = P_{\text{hse}} / ((\gamma-1) \cdot \rho_{\text{hse}})$ to the device:

```
void alloc_cooling_ref(const std::vector<double>& e_ref_per_row);  
//  $\rightarrow$  cudaMalloc(d_e_ref_y, ny * sizeof(double))
```

This runs whether or not tau_cool > 0 — the memory is tiny (a few KB), and --cool-tau <s> becomes a runtime switch rather than a recompile.

5. Newton Cooling

5.1 The update

For every cell, each step:

$$e \leftarrow e + (e_{\text{ref}}(y) - e) \cdot (1 - \exp(-dt / \tau_{\text{cool}}))$$

e_{int} only — not ρ , not v . Mass conservation and HSE are exactly preserved. When $v = 0$ the cooling pulls P back toward P_{hse} , $\rho \cdot g$ is unchanged, HSE reassembles. In active convection the upflow carries high-entropy fluid up; cooling bleeds the excess entropy off at the top-heavy side of each cycle.

5.2 Why not real radiation

Proper radiative sink: $de/dt = -\nabla \cdot F_{\text{rad}} + Q_{\text{vis}}$, with $F_{\text{rad}} = -16\sigma T^3 / (3\kappa\rho) \cdot \nabla T$. That needs implicit BE. `cart_ale2` has no such solver (`radial1d` does). Porting one is a ~1-week job — too expensive for a pipeline-validation round.

Newton cooling is the Stein-Nordlund 1980s / Kupka-Muthsam pre-CO⁵BOLD standard simplification: **same sink physics**, letting convection equilibrate. The difference is τ_{cool} does not depend on κ and is spatially uniform. That is enough for "establish steady convection and validate pipeline", not enough for "report quantitative v_{conv} ".

5.3 Implementation

Kernel:

```
__global__ static void k_cale2_newton_cool(
    double* e_int, const double* e_ref_y,
    double alpha, int nx, int ny)
{
    int ic = blockIdx.x*blockDim.x + threadIdx.x;
    int jc = blockIdx.y*blockDim.y + threadIdx.y;
    if (ic ≥ nx || jc ≥ ny) return;
    int idx = ic*ny + jc;
    e_int[idx] += (e_ref_y[jc] - e_int[idx]) * alpha;
}
```

Called inside `step()` after Lagrangian + remap and before `step_count++`:

- **Why that position.** After remap the cell state is finalised; doing a closed-form relaxation on e is trivial, no risk of colliding with any other operator.
- $\alpha = 1 - \exp(-dt/\tau)$. For $dt \ll \tau$ this is linear $\approx dt/\tau$, for $dt \gg \tau$ it saturates at 1 (i.e. e is clamped to e_{ref} — a stiff isothermal boundary).

5.4 How to pick τ

- $\tau_{\text{cool}} \ll \tau_{\text{dyn}}$ (say $0.1 \tau_{\text{dyn}}$): cooling is faster than acoustic overturn $\rightarrow$ cells are flattened before they can form $\rightarrow$ convection is suppressed.
- $\tau_{\text{cool}} \gg \tau_{\text{dyn}}$ (say $10 \tau_{\text{dyn}}$): cooling is too slow, heat accumulates, behaviour is close to the no-cooling case.
- $\tau_{\text{cool}} \approx \tau_{\text{dyn}}$: overturn time matches cooling time $\rightarrow$ textbook saturated convection. This is the range Muthsam+ 2010 CO⁵BOLD uses. We verified that $\tau = 500$ s and $\tau = 2700$ s converge to essentially the same steady state (§6.2).

6. Numerical Results

6.1 Common configuration

--nr 128 --ntheta 128 --cfl 0.4 --ppm --remap-order 2 --bc-x periodic --bc-y reflect
--tend 15000, slab = envelope $r/R \in [0.85, 0.99]$ of the $1 M_{\odot}$ ZAMS profile.

6.2 Energy / velocity steady state

Setting	t_end	steps	KE [erg]	v_max [cm/s]	E drift	Note
No cooling	15000 s	9061	7.22×10^{29}	1.44×10^6	0.12 %	KE still rising at $5 \tau_{\text{dyn}}$
$\tau_{\text{cool}} = 2700$ s ($\approx \tau_{\text{dyn}}$)	15000 s	9008	5.40×10^{29}	1.29×10^6	0.09 %	Clear saturation
$\tau_{\text{cool}} = 500$ s ($\approx 0.2 \tau_{\text{dyn}}$)	15000 s	9013	5.52×10^{29}	1.30×10^6	0.07 %	Same as $\tau = 2700$

Observations: - Cooling takes 25 % off KE and 15 % off v_{max} , but the steady state still sits at $\text{Mach} \approx 0.4 \times c_{\text{s}}(\text{top})$. - Dropping τ from τ_{dyn} to $0.2 \tau_{\text{dyn}}$ moves almost nothing — cooling has saturated the sink, and the bottleneck is elsewhere (see §6.3). - E drift is under 0.15 % in every case; mass conservation holds to ten digits.

6.3 Why v_{max} is still $\sim 200\times$ the MESA MLT value

MESA at $1 M_{\odot}$ ZAMS predicts $v_{\text{conv}} \approx 5000 - 6000$ cm/s in the same r/R range. Our box sits at 1.3×10^6 cm/s. **The reason is no bottom enthalpy flux.**

In MESA the $L_{\odot}$ is produced by nuclear burning and advected outward; inside the convection zone $L_{\text{conv}} + L_{\text{rad}} \approx L_{\odot}$, so the convection is **flux-driven**. Our box bottom is a reflective wall — no L_{bottom} , no source.

The only driving is the buoyancy release from the initial δs perturbation plus the ongoing compression $\leftrightarrow$ expansion exchange, no steady-state flux.

Newton cooling is a sink, not a source. In steady state $\Sigma \text{ cooling} \cdot dV \approx 0$ (IE barely decreases); the convection runs inside a "near-adiabatic compression $\leftrightarrow$ near-adiabatic expansion" loop whose velocity magnitude is set by $c_s \times (H_P/Ly)^{1/2} \sim \text{Mach } 0.1$, not by the stellar MLT value.

To bring v_{max} down to the MESA scale we must add a **bottom $L_\odot$ source term**:

$$\partial e / \partial t |_{y=0} = F_\odot / dm_{\text{bot}} = (L_\odot / (4\pi R_{\text{bot}}^2)) / (\rho_{\text{bot}} \cdot dy)$$

$\sim 1 \times 10^4 \text{ erg/(g}\cdot\text{s)}$ heating. In steady state $\text{cooling}(\text{top}) = \text{heating}(\text{bot}) = L_\odot$, the convective flux gets pinned, and v_{conv} settles on the MLT value. **Queued for the next session.**

6.4 Pipeline-validation metrics

These are the numbers that say **the pipeline is correct**, independent of the velocity-magnitude question:

Quantity	No cooling	$\tau = 2700$	Verdict
Mass conservation	10 digits	10 digits	[OK] swept remap clean
HSE residual at $t=0$	$1e-13$	$1e-13$	[OK] slab extractor OK
Convection cells form	yes	yes	[OK] bottom seed propagates
Steady state reached	$\sim 5 \tau_{\text{dyn}}$	$\sim 2 \tau_{\text{dyn}}$	[OK] cooling accelerates equilibration
VTK frames	31 / 15000 s	31 / 15000 s	[OK] VRAM I/O OK

7. Honest Limitations

Logged here so nobody reruns into them:

7.1 v_{max} cannot auto-match MESA MLT

Reason in §6.3: missing bottom flux. ****Do not try to pull v_{max} down by shrinking τ_{cool} **** — cooling is a sink, the source is missing, and pressing harder on the sink drags IE with it and produces a cold artefact.

7.2 Ideal $\gamma = 5/3$ is fixed

MESA reports $\Gamma_1 \approx 1.6655$ in the convection zone, error $< 0.05\%$. If the slab is extended into the shallow envelope ($r/R > 0.99$) partial H ionisation drops Γ_1 to ~ 1.2 — ideal γ is no longer safe. **Keep slab $r/R < 0.995$** for this solver; anything above needs a real EOS.

7.3 No κ dependence

No κ table $\rightarrow$ cooling is spatially uniform $\rightarrow$ the optically-thin $\leftrightarrow$ optically-thick transition (photosphere) cannot be represented. Our slab stops at $r/R = 0.99$, where $\tau \gg 10^3$ (deep optically thick) and the no- κ cooling is a reasonable stand-in. Anything shallower needs a κ -aware radiation module.

7.4 Reflective bottom vs periodic bottom

--bc-y periodic would loop bottom plumes back from the top, giving a non-physical dual-convection-cell crash. **Keep --bc-y reflect**, both top and bottom. If we later want a transparent bottom, we need new BC kernels with a sponge layer.

8. Reproducibility

```
# Step 1 (once): MESA 1 M $\odot$   $\rightarrow$  ZAMS. Already done.
#  $\rightarrow$  /tmp/mesa_work_1Msol/LOGS/profile5.data

# Step 2: slab exporter
python3 scripts/make_local_convection_slab.py \
    /tmp/mesa_work_1Msol/LOGS/profile5.data \
    /tmp/slab_envelope.txt \
    --r-lo 0.85 --r-hi 0.99 --ny 128 --lx-over-ly 2.0

# Expected stderr tail:
# HSE resid (rel): median 6.01e-14, p90 3.51e-13, max 3.98e-09
#  $\tau_{\text{dyn}}$  (Ly/c_s_top): 2.698e+03 s

# Step 3: cart_ale2 run (Newton cooling,  $\tau \approx \tau_{\text{dyn}}$ )
./build/stellar2d --solver cart_ale2 --test local_convection \
    --ic-slab /tmp/slab_envelope.txt \
    --nr 128 --ntheta 128 \
    --bc-x periodic --bc-y reflect \
    --remap-order 2 --ppm \
    --cfl 0.4 --tend 15000 \
    --vtk-dt 500 --diag-interval 200 \
    --frame-buffer --cool-tau 2700

# 3 s wall time, 31 frames, KE sat  $\sim 5.4e29$ ,  $v_{\text{max}} \sim 1.3e6$ .
```

8.1 Deeper / shallower slab

Deep convection ($r/R \in [0.70, 0.95]$) or close to the surface ($r/R \in [0.95, 0.995]$):

```
python3 scripts/make_local_convection_slab.py ... --r-lo 0.70 --r-hi 0.95
# Ly grows ( $\sim 25\%$   $R_{\text{sun}}$ ),  $\tau_{\text{dyn}}$  grows, scale --tend accordingly.
```

8.2 No-cooling control

Drop `--cool-tau`, everything else the same $\rightarrow v_{\text{max}} 1.44e6$ (see §6.2).

8.3 Aggressive cooling

`--cool-tau 500` $\rightarrow$ almost identical to $\tau = 2700$ (§6.2). Cooling saturated.

9. Future Work

9.1 Add a bottom enthalpy-flux source (the immediate next step)

Minimal: extend `k_cale2_newton_cool` with a bottom-row source:

```
if (jc == 0) {           // bottom row
    e += heat_rate * dt; // heat_rate = L⊙ / (4π R2 · ρbot · dy)
}
```

- Use $L_{\odot} = 3.83 \times 10^{33}$ erg/s / (cell volume $\times \rho_{\text{bot}}$) for the coefficient.
- In steady state bottom heating = top cooling (the two must be decoupled — probably by limiting cooling to the top few rows too).
- Expected: v_{conv} drops to the MESA MLT scale.

Work estimate: ~2 hours.

9.2 Spatially-varying τ_{cool}

Nordlund-Stein 1998: $\tau_{\text{cool}}(y) = \tau_{\text{top}} \cdot \exp(-y / H_{\text{cool}})$, confining cooling to the upper layer. Required for any serious surface granulation work.

Work estimate: ~1 day.

9.3 `cart_ale3 = cart_ale2 + helm + κ + implicit BE radiation`

The "real" 2D stellar-convection solver, peer to CO⁵BOLD. Work estimate: 2–3 weeks. Not on the current path. CLAUDE.md rules say new solver = new files; do not mutate `cart_ale2`.

9.4 Compare surface granulation with observations

radial1d ZAMS + `cart_ale2` local_convection $\rightarrow v_{\text{conv}}(y)$, rms $\delta\rho(y)$, cell-size distribution. Compare to solar MDI/HMI granulation spectra. This is the increment that MESA and GYRE cannot produce.

10. Cross-References

- [radial1d_mesa_tier2_pk_2026-05-03.md](#) — Tier-2 PK report (radial1d's 1D ceiling).
- [radial1d_session_2026-05-02_03_journal.md](#) — the prior session's full narrative.
- [cart_ale2_design.md](#) — cart_ale2 architecture and applicability.
- [pitfalls.md](#) — P30, P31 on cart_ale2 periodic-BC subtleties (not repeated here).

11. Commits Preview (pending)

- `scripts/make_local_convection_slab.py` — new.
 - `src/gpu/cart_ale2_solver.cuh` — new declarations: `init_local_convection`, `tau_cool`, `d_e_ref_y`, `alloc_cooling_ref`, `apply_cooling`.
 - `src/gpu/cart_ale2_solver.cu` — new IC + kernel + `apply_cooling` call site.
 - `src/main.cpp` — five new flags: `--test local_convection`, `--ic-slab`, `--slab-perturb`, `--slab-seed-k`, `--cool-tau`.
 - `docs/cart_ale2_local_convection_2026-05-03.md` — this document.
 - `docs/cart_ale2_design.md` — one line added to the applicability list.
-

Appendix A — Session Pitfall Log

A.1 Naïve log-interp of MESA (ρ , P) breaks HSE

Symptom. Slab file HSE residual median 0.8 %, max 2.5 %. Feeding it to `cart_ale2` produces $|v|_{\max} \approx 1 \times 10^5$ cm/s at $t = 0$ — which is numerical HSE disequilibrium, not real convection.

Root cause. MESA's P balances $g(r) = GM/r^2$; flattening to constant g_y breaks the balance.

Fix. Preserve MESA entropy $s(y)$, anchor at the top, and trapezoidally integrate $dP/dy = -\rho g$ with the entropy constraint (§3.2). HSE residual drops to 6e-14.

Regression test. `make_local_convection_slab.py` prints HSE `resid (rel)` to `stderr`. Watch that the median stays below 1e-10.

A.2 `e_ref(y)` is per-row, not per-cell

First sketch tried to look up `e_ref(Yc)` in the cooling kernel, which means the kernel needs the y-coordinate grid. Unnecessary overhead. Replaced with a flat `e_ref_per_row[ny]` device array; kernel indexes by `jc` — one load, no branch.

A.3 `tau_cool` must be set after the IC runs

`init_local_convection` calls `alloc_cooling_ref` which allocates `d_e_ref_y`. If the CLI dispatch sets `cale.tau_cool = ...` before the IC, the later IC call still has to run, but the sequence is fragile (re-init would wipe the setting). **Correct order:** `init_local_convection` → `set tau_cool`. `main.cpp` does this in the right order today.

Closing

Bridgehead reached. The pipeline `radial1d` 1D ZAMS → slab extractor → `cart_ale2` 2D local-convection box runs in 3 s for $5.5 \tau_{\text{dyn}}$, preserves mass to ten digits, builds convection cells, and saturates under Newton cooling to a sensible — if still un-MLT-calibrated — steady state.

What is not yet reached. v_{conv} is $200\times$ larger than MESA MLT because we have no bottom $L\odot$ source. That is an explicitly bounded next step, not a vague TODO.

Where this sits on the strategic map. - Past 48 h: `radial1d` + MESA 1D validation (Tier-2 PK, 2.5 %-level accuracy, 1D ceiling). - Today: the 1D-ZAMS → 2D-IC pipeline is in place. - Next: feed the 1D $L\odot$ into the 2D bottom too (enthalpy flux), v_{conv} should snap to MLT values automatically. - After that: either `cart_ale3` (real EOS + radiation) or pushing `cart_ale2` to its surface-granulation limits.

The full pipeline today: **MESA produces ZAMS → radial1d polishes 1D → slab extractor flattens → cart_ale2 runs convection.** One line, four links, every link reproducible from the commands in §8.